15: Inserting Data with Prepared Statement

A **PreparedStatement** is an interface in Java that extends the Statement interface, used to execute precompiled SQL queries.
Unlike a regular Statement, where SQL queries are created and executed directly, a PreparedStatement allows you to write SQL queries with placeholders (known as parameters) and inject values dynamically at runtime.

👉 **Parameterized Queries:**
Prepared statements allow you to use placeholders (e.g., ?) in your SQL query, and these placeholders will be replaced with actual values at runtime.
Such queries are called parameterized queries because the data is injected into the query at runtime instead of being hardcoded in the SQL query itself.

👉 **Key Features of PreparedStatement:**

- **Parameterized Queries:** It allows you to define placeholders (?) within the SQL query where actual values will be substituted at runtime.
- **Improved Security**: Since the values are set using the set methods, it protects against SQL injection attacks.
- **Performance:** Prepared statements are often **precompiled**, which means that if the **same query is executed multiple times**, it can be executed faster since the query plan does not need to be recreated every time.
- **Type-Specific Methods:** It provides methods like setString(), setInt(), setDouble(), etc., for different data types, ensuring type safety when inserting data.

👉 **Steps for Inserting Data with Prepared Statement:**

1. **Create a PreparedStatement:** Write an SQL query with placeholders (?) for the values to be inserted. Create a PreparedStatement object with this query.

```
String query = "INSERT INTO studentinfo (id, sname, sage, scity) VALUES (?, ?, ?, ?)";
PreparedStatement pstmnt = connect.prepareStatement(query);
```

➔ A PreparedStatement object is created by passing the query to connect.prepareStatement(query).

2. **Fetching Data at Runtime:** Values can be captured dynamically at runtime using input from the user using the Scanner class.

```
System.out.println("Please enter the following details to be stored in DB");
Scanner scan = new Scanner(System.in);

System.out.println("Enter your id");
 Integer id = scan.nextInt();

 System.out.println("Enter your name");
 String name = scan.next();

 System.out.println("Enter your age");
  Integer age = scan.nextInt();

  System.out.println("Enter your city");
  String city = scan.next();
```

3. **Setting the Parameters:** Prepared statements have various set methods (such as setString, setInt, setDouble, etc.) depending on the type of data you are injecting into the query.
   You pass two arguments to the set method:

   - The index of the placeholder in the query (1-based indexing).
   - The variable name or reference to the value being inserted.

```
pstmnt.setInt(1, id);
pstmnt.setString(2, name);
pstmnt.setInt(3, age);
pstmnt.setString(4, city);
```

4. **Execute the Query:** Call executeUpdate() to insert the data into the database.

```
int rowAffected = pstmnt.executeUpdate();
// process the result
if (rowAffected == 0) {
    System.out.println("Unable to insert the data");
} else {
    System.out.println("Data Inserted Successfully!");
}
```

5. **Close Resources:** Always close the PreparedStatement and Connection objects to free up resources.

```
catch (SQLException e) {
    e.printStackTrace();
} catch (Exception e) {
    e.printStackTrace();
}
finally {
    //close the resources
    try {
        JdbcUtil.closeConnection(connect, pstmnt);
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```